

FUSE-NF: Constructing an Elegant Normal Form for Natural Language Semantics

Ben Goertzel

May 26, 2025

NOTE: I used an LLM to help generate some examples and summaries here, interspersed throughout the document.

1 Introduction

Building robust, scalable AGI systems based on the ideas involved in the Hyperon and PRIMUS architectures requires representations that bridge symbolic precision and statistical generalization. Holman’s Elegant Normal Form (ENF), used in the MOSES program learning subsystem, is one example of this bridging. ENF provides a canonical, minimal DAG for Boolean expressions via a small set of confluent rewrite rules, enabling nearly backtracking-free SAT solving and also simplifying statistical and inferential analysis of populations of expressions. ENF has been extended in the OpenCog project to include fairly general programs, well beyond its roots in Boolean expressions. Here we extend the ENF idea to natural language structures, introducing the notion of a Semantic Elegant Normal Form (SENF) and its fuzzy variant FUSE-NF. SENF lifts ENF’s confluence, termination and semantic-preservation guarantees to tree representations of natural language syntax and semantics, while FUSE-NF further collapses common idioms and role variants via statistical pattern mining across a large SENF corpus.

The core of FUSE-NF is a pipeline that takes each sentence through two parallel parses—a CCG (Combinatory Categorical Grammar) -derived lambda tree and an LLM-generated predicate logic tree – cross-links them, and then applies data-driven consolidation rules (frequent-subtree mining, role-filler clustering, mutual information grouping, paraphrase alignment and autoencoder analysis). Background Probabilistic Logic Network inference injects KB-derived entailments, and causal discovery labels edges as causal or correlational. The resulting fuzzy-SENF graph is encoded with a causal-aware graph transformer whose attention is biased and gated by edge types and confidences. This unified representation supports efficient symbolic search, probabilistic evolutionary learning

(e.g. within MOSES), and causal reasoning, making it a powerful foundation for Hyperon-based AGI.

The use of CCG and predicate logic here is for convenience, and these could be replaced by other approaches to producing syntactic and logic-semantic tree or dag structures from natural language sentences, without necessarily affecting the overall methodology.

On the whole, this should be considered a rough concept paper intended to indicate a direction – the ideas given here could be developed in a lot of particular ways and the specifics given here probably are not optimal!

2 Semantic Elegant Normal Form (SENF): Definition and Properties

Holman’s Elegant Normal Form (ENF) was originally devised to tame Boolean expressions: by applying a small set of confluent, terminating rewrite rules—merging redundant branches, eliminating impossible guards, and subsuming nested tests—it turns any formula into a unique, minimal constraint-DAG that preserves exactly the same truth-function but admits trivial, almost backtracking-free search in practice.

2.1 An Example of Good Old Boolean ENF

For the reader unfamiliar with ENF we will give a brief example here – just for general background; the details are quite different from SENF and not really needed for understanding how SENF works.

Consider the Boolean formula

$$E = (x \wedge y) \vee (x \wedge \neg y).$$

We will apply Holman’s eight local rewrite rules to reach its unique Elegant Normal Form (ENF), which in this case is simply x .

1. Constraint-Tree Construction.

Interpret each disjunct as a leaf guarded by its conjunction of literals:

$$E \longrightarrow \text{OR}\left(\underbrace{\{x = 1, y = 1\}}_{\text{guard } G_1}, \underbrace{\{x = 1, y = 0\}}_{\text{guard } G_2}\right).$$

2. Guard Pull-Up.

Since both children share $x = 1$, we apply the MERGE-GUARDS rule:

$$\text{OR}(G_1, G_2) \longrightarrow \{x = 1\} \wedge \text{OR}(\{y = 1\}, \{y = 0\}).$$

3. Tautology Elimination.

The subexpression $\text{OR}(\{y = 1\}, \{y = 0\})$ is always true, so the TAUTOLOGY rule yields:

$$\{x = 1\} \wedge \text{True} \longrightarrow \{x = 1\} \wedge \text{True}.$$

(Here we recognize and drop the tautology.)

4. True-Conjunction Elimination.

Conjoining with True is a no-op, so the TRUE-ELIM rule gives:

$$\{x = 1\} \wedge \text{True} \longrightarrow x.$$

The expression x is now in ENF: a unique, minimal form that preserves the exact truth-function of the original E .

2.2 Extending ENF

The same idea Holman developed for Boolean expressions extends naturally to far richer programmatic settings: whether you’re normalizing higher-order rewrite rules in the MeTTa calculus, pruning and merging guarded branches in an intermediate language like MeTTa-IL, or folding correlated subtests into meta-guards in a population of evolving programs, the core pattern remains the same. By collapsing all “obvious” simplifications and encoding only the irreducible semantic alternatives, these generalized ENF forms can be used to power a variety of AI tools ranging from lightning-fast SAT solvers to scalable symbolic-probabilistic program synthesis, evolutionary learners like MOSES in Hyperon, or perhaps even causal graph-aware transformers that combine logic, tree structure, and learned commonsense in a single compact representation.

Intuitively, you can think of an Elegant Normal Form as the “cleaned-up” core of any expression or program, where all the obvious redundancies and detours have been swept away but the true meaning is left intact. You start with a few simple, local rewrite rules—things like “if two branches do the same test, merge them,” or “drop any condition that can never fire”—and you apply them in any order until nothing more changes. Because the rules are designed to be confluent (different application orders lead to the same result) and terminating (each step makes the representation strictly simpler), you end up with one unique, minimal tree or graph that still computes exactly the same function. In this form, every node corresponds to a genuinely necessary decision or operation, and every branch represents a real semantic alternative, so measuring differences between two ENF trees really tracks how their behaviors differ. That combination of canonicity, minimality, and semantic faithfulness is what makes an Elegant Normal Form so powerful for both symbolic reasoning and statistical modeling.

Following this intuition, what I mean by a *Semantic Elegant Normal Form* (SENF) is a canonical, minimal, semantics-preserving representation of a natural language sentence’s meaning as a rooted, directed acyclic graph (possibly

with limited re-entrancies). Crudely put, SENF generalizes Holman’s Elegant Normal Form (ENF) for Boolean constraint trees to arbitrary semantic operator alphabets (syntax trees, predicate-logic trees, or both), in a manner intended to be helpful for commonsense reasoning and related applications.

3 Key Properties of SENF

The practical meaning of SENF will become clear once we start going through the process of putting a sentence in SENF. However, first let’s review some of the abstract properties that we want the SENF representation of a sentence to have.

- **Confluence and Termination.** There is a finite set of local rewrite rules (pruning, merging, collapsing) that are *terminating* (each reduces a well-founded weight) and *locally confluent*. It then mathematically follows that every sentence has a unique SENF.
- **Semantics Preservation.** All rewrites preserve the intended meaning (truth-conditions, predicate-argument structure), so $\text{SENF}(s)$ is semantically equivalent to the original sentence s . (This may never be totally true given the vagaries of natural language semantics, but we need it to be close enough....)
- **Canonical Minimality.** SENF eliminates redundant or vacuous structure (e.g. empty modifiers, duplicate branches) and merges recurring patterns into single meta-nodes, producing a relatively minimal graph.
- **Locality of Edits.** Every local rewrite or consolidation step affects only a bounded, well-defined part of the graph, enabling a tight relationship between graph-edit distance and semantic distance.
- **Uniqueness.** Regardless of the order in which rewrites are applied, every input sentence maps to the same SENF graph.
- **Extensibility.** ENF for Boolean constraint trees emerges as a special case of SENF when the operator alphabet is $\{\wedge, \vee, \neg\}$ and guards are Boolean literals. SENF extends this to:
 - *Syntactic operators* (CCG combinators, function words).
 - *Semantic predicates* (FOL atoms, event predicates).
 - *Fuzzy consolidations* learned from data (role merges, idiom collapses).

3.1 Relation to ENF

As reviewed above, ENF (*Elegant Normal Form*) is a canonical form for Boolean expressions under a finite set of confluent tree-rewrites that yield a unique minimal constraint DAG. SENF retains the same algebraic theory of rewriting:

- *Rewrite rules* are extended to cover semantic operators (predicate merges, quantifier flattening, modifier pruning).
- *Weight measures* now account for node counts plus sizes of guard or role-sets.
- *Confluence proofs* follow the same well-founded order argument as in ENF.

Thus, SENF can be seen as "ENF lifted to the semantic domain," guaranteeing that small structural edits correspond to small semantic changes, and providing a unique, compact representation ideal for downstream statistical and neural processing.

3.2 From SENF to FUSE-NF

Due to the differences between natural language and formal language, SENF as described above does not provide sufficient regularization of natural language sentences for many practical applications. So we also develop an additional stage of normalization called fuzzy SENF or FUSE-NF, which provides the same DAG form for sentences that have close but not so precisely similar meanings. The steps to get from SENF to FUSE-NF deviate a bit further from the foundation in ENF, but they build on an SENF foundation.

4 The Process of Getting from Sentence to SENF to FUSE-NF

FUSE-NF produces a single Semantic Elegant Normal Form (SENF) graph that merges two parallel structured interpretations of a sentence: a CCG-derived semantic tree and an LLM-derived predicate-logic tree. These are cross-linked, then consolidated via statistical pattern mining, enriched with background PLN inferences, and finally encoded by a causal-aware transformer. The result is a unique, minimal, semantics-preserving representation that supports both precise logical reasoning and robust statistical generalization.

4.1 CCG Parse and Logic Parse

First, FUSE-NF obtains two independent views of sentence structure. A CCG parser produces a syntactic derivation together with a compositional lambda-term; this lambda-term is beta-normalized and eta-expanded to expose all functional applications. In parallel, we prompt a large language model to generate a standalone first-order logic (FOL) formula capturing the same meaning. That formula is cleaned by stripping lambda binders, fixing argument order to a pre-defined role schema, and consistently alpha-renaming its variables. Having both views allows us to align syntactic roles with logical arguments and to recover meaning that might be missed by either parser alone.

4.2 Building the Initial SENF Graph

From the CCG lambda-term we construct the *syntax branch* of the SENF graph: each top-level predicate becomes a node whose children represent its argument slots and any unary modifiers (tense, aspect, modality). Simultaneously, each atomic predicate in the FOL formula is turned into a node in the *logic branch*, with children for each argument term. We then add undirected

alignment edges between corresponding argument slots in the two branches—for example, linking the CCG "Agent" child of a `go-to` node to the first argument of the FOL atom `Go(e, Agent, CoAgent, Destination)`. This cross-linking ensures that word-level structure and formal predicate structure mutually inform one another in downstream processing.

4.3 Pattern Mining for Consolidation Rules

To collapse recurring surface variations and idioms into compact meta-nodes, we mine a large corpus of SENF graphs for statistically robust patterns. This could be done in a variety of different ways, we outline here several approaches that seems likely workable. Experimentation with multiple approaches in combination may be valuable.

4.3.1 Frequent Subtree Mining

Here we enumerate all rooted subtrees up to a fixed size threshold in each SENF graph and count how many sentences contain each pattern. Subtrees exceeding a minimum support threshold reveal common semantic units—such as "go-to with two participants"—that should be treated as single meta-nodes.

4.3.2 Role-Filler Distribution Clustering

For every predicate-slot (e.g. `go-to.Agent` or `Agent2`), we collect the set of fillers across the corpus and embed them in a vector space (using word or subtree embeddings). Clustering these embeddings reveals when two slots share indistinguishable distributions of fillers, indicating they fulfill the same semantic role and can be merged.

4.3.3 Mutual-Information Grouping

We could construct a binary feature matrix indicating which subtrees occur in which sentences, then compute pairwise mutual information between features. Pairs with very high MI but moderate individual support almost always co-occur, so they are excellent candidates for consolidation into a single feature.

4.3.4 Paraphrase-Based Alignment

Given a set of sentence pairs known to be paraphrases (e.g. via asking an LLM to rate if they are paraphrases or not?), we align their SENF graphs (via tree-edit or soft matching) and record which subtrees and roles consistently map to each other. These alignments validate which structural elements can be unified without semantic loss.

4.3.5 Autoencoder Analysis

We vectorize each SENF graph by its feature counts and train a shallow autoencoder with a low-dimensional bottleneck. Input features whose activations are tied together in the encoder weights indicate clusters of subtrees that serve interchangeable semantic functions—another source of consolidation rules.

4.4 Applying Consolidation Rules

With a validated rule set in hand – obtained by whatever combination of methods – we then transform each new SENF graph as follows:

- **Slot Merges:** Whenever two slots (e.g. **Agent** and **CoAgent**) have been deemed equivalent, we replace them with a single (e.g. **Participants**) slot whose filler is the union of the originals.
- **Modifier Pruning:** If a modifier (e.g. **Manner=together**) is redundant given other slots (e.g. **Participants** of size > 1), we delete that modifier node.
- **Subtree Collapses:** High-MI subtree pairs are replaced by a new meta-node representing their combined semantics.
- **Role Renames:** Slots aligned by paraphrase mining or clustering are uniformly renamed to the consolidated label.
- **Iteration:** Optionally, we re-mine the transformed corpus of fuzzy-SENF graphs to discover higher-order patterns and refine rules.

4.5 Background PLN Inference

Next we feed the FOL branch of the SENF graph into a Probabilistic Logic Network over a background knowledge base to derive additional entailments and probabilistic inferences. Each new fact or implication—tagged with its PLN confidence—is injected back into the SENF graph as either a new node or a directed edge, thereby enriching the representation with background knowledge.

4.6 Constructing a Causal-Association Graph

To distinguish causal relationships from mere correlations, we can label inferred rules as causal or associational (via temporal markers or causal discovery tests) and insert:

- *Directed edges* for validated causal implications, oriented from cause to effect.
- *Undirected edges* for high-confidence correlational associations.

Each edge carries both its type and a learned confidence weight for downstream gating.

5 Transformer Integration

An interesting follow-on step could be to use FUSE-NF in the context of training a transformer neural network, following the rough fashion of tree transformers.

5.1 Graph Feature Construction

Each node in the final FUSE-NF graph includes a composite feature vector combining:

- The consolidated FUSE-NF label (predicate-slot or meta-node).
- Any associated lexical or subtree embedding.
- Logic annotations such as variable identifiers or PLN confidences.

Edges are annotated with learned type embeddings for alignment, causal-forward, causal-backward, or correlational.

5.2 Attention Biases and Gating

In the graph transformer’s self-attention mechanism, we add learned bias terms based on edge types, and we gate each message by the edge’s confidence weight, ensuring that high-confidence causal links have greater influence.

5.3 Training Objectives and Inference

The model is trained on the main downstream task (e.g. question answering, natural language inference), augmented by auxiliary objectives such as predicting injected facts and maintaining robustness to simulated interventions (masking incoming causal edges). At inference time, we can simulate ”do-operations” by masking causal inputs to a node and letting the model recompute its effects, supporting genuine causal reasoning.

6 Illustrative Example

Input Sentence “After finishing her homework, Maria quickly drove to the store with John because it was raining heavily.”

6.1 CCG Parsing and Lambda-Term

We run a CCG parser to get both a syntactic derivation and a compositional lambda-term. After beta-normalization and eta-expansion, we obtain something like:

```
lambda e. Drive(e, Agent=Maria, CoAgent=John, Destination=Store,
    Manner=quickly)
  ^ After(e1, Finish(e1, Agent=Maria, Theme=Homework))
  ^ Cause(e2, Rain(e2), e)
  ^ Manner(e2, heavily)
  ^ Past(e), Past(e1), Past(e2)
```


6.2 LLM-Based Predicate-Logic Interpretation

In parallel, we prompt an LLM to produce a standalone FOL formula:

```
ThereExists e,e1,e2 [  
  Finish(e1, Maria, Homework)  
  ^ Drive(e, Maria, John, Store, quickly)  
  ^ Rain(e2)  
  ^ Cause(e2, e)  
  ^ Manner(e2, heavily)  
  ^ Before(e1, e)  
  ^ Past(e1) ^ Past(e2) ^ Past(e)  
]
```

We canonicalize argument order (Agent, CoAgent, Destination, Manner), strip lambdas, and alpha-rename variables.

6.3 Building the Initial SENF Graph

We construct two parallel branches and cross-link them:

6.3.1 Syntax Branch (from CCG)

```
Drive  
|--- Agent      : Maria  
|--- CoAgent    : John  
|--- Destination: store  
|--- Manner     : quickly  
    |--- Temporal  
        |- AfterFinish  
            |- Agent : Maria  
            |- Theme : homework
```

```
Cause  
|--- CauseOf : Rain  
|--- EffectOf: Drive  
    |- Manner: heavily
```

6.3.2 Logic Branch (from FOL)

```
Finish  
|--- Agent : Maria  
|--- Theme : homework  
  
Drive  
|--- Agent      : Maria  
|--- CoAgent    : John  
|--- Destination: store
```

```
|--- Manner      : quickly
    |--- Temporal : Past
```

```
Rain
    |--- Manner : heavily
```

```
Cause
    |--- CauseOf : Rain
    |--- EffectOf: Drive
```

6.3.3 Cross-Linking

We add undirected alignment edges between corresponding slots, e.g. every CCG ‘Agent’ node is linked to the FOL atom’s first argument, synchronizing the two views.

6.4 Applying Consolidation Rules

Using pattern-mined rules (from frequent-subtree, MI, role-clustering, paraphrase alignment, autoencoder):

- **Merge Agent+CoAgent ? Participants:** combine ‘Maria’ and ‘John’ into {Maria, John}.
- **Prune redundant ‘Manner’ in Cause branch:** drop ‘Manner=heavily’ under ‘Cause’, keep it under ‘Rain’.
- **Collapse AfterFinish subtree:** replace ‘AfterFinish’ + its children by a single node ‘AfterFinishHomework’.
- **Rename roles consistently:** ensure both branches use the same slot names.

6.5 FUSE-NF Graph

After consolidation and pruning, the merged SENF graph is:

```
Drive
    |--- Participants : {Maria, John}
    |--- Destination : store
    |--- Manner       : quickly
    |--- Temporal     : AfterFinishHomework
```

```
Rain
    |--- Manner : heavily
```

```
Cause
    |--- CauseOf : Rain
    |--- EffectOf: Drive
```

6.6 Background PLN Inference

We feed the FOL branch into a Probabilistic Logic Network over a KB. It infers, for example:

- `Completed(Homework)` with confidence 0.95
- `Motion(e)` from `Drive(e)` with confidence 0.90
- `Wet(store)` from `Rain(e2)` with confidence 0.85

These are injected as new nodes:

```
Completed
  |- Homework
```

```
Motion
  |- Drive(e)
```

```
Wet
  |- store
```

6.7 Constructing the Causal-Association Graph

We insert:

- *Directed causal edges:* `Rain→Drive`, `FinishHomework→Drive`.
- *Undirected correlational edges:* `Participants↔Manner(quickly)`.
- Edge confidences from mining or PLN.

6.8 Graph Transformer Integration

- **Node features:** consolidated label, lexical embedding, PLN confidence.
- **Edge embeddings:** types (syntax-logic alignment, causal-forward, causal-backward, correlation).
- **Attention / gating:** add type biases, gate by edge weight.
- **Training:** combine main task loss with auxiliary fact-prediction and intervention objectives.
- **Inference:** support ‘do(Rain)’ by masking incoming edges to ‘Rain’ and recomputing.

This illustrates roughly how one might apply FUSE-NF to a richly structured sentence, yielding a unique, minimal, semantically grounded SENF graph for downstream reasoning and learning.

7 Conclusion and Future Work

Here we have introduced FUSE-NF, a pipeline for deriving a Fuzzy Semantic Elegant Normal Form that unifies CCG-based syntactic parses and LLM-generated predicate-logic interpretations into a single, confluent graph. By mining statistical patterns across large SENF corpora—via frequent-subtree enumeration, role-filler clustering, mutual information grouping, paraphrase alignment and

autoencoder analysis—we automatically discover consolidation rules that collapse idiomatic variants into compact meta-nodes. Background PLN inference enriches the graph with entailments, while causal-association edges distinguish true causal dependencies from mere correlations. A graph transformer with edge-type biases and gating then encodes this enriched representation, yielding a robust foundation for downstream tasks within a variety of Hyperon-based AI and AGI systems.

Looking forward, one could extend FUSE-NF in several directions. One could explore fully unsupervised rule discovery using probabilistic grammars and neural autoencoders to further reduce reliance on (LLM-based) paraphrase annotations. One could integrate multimodal knowledge—linking visual scene graphs or action models into the SENF graph—to support grounded reasoning. One could develop adaptive consolidation, where the system refines its pattern-mined rules online as it encounters new domains or specialized vocabularies (e.g. using PLN inference along with other methods). Finally, it will be important to evaluate the end-to-end pipeline on a variety of practical tasks, to measure how much FUSE-NF enhances both symbolic precision and statistical generalization in practice.